Week 1: Reading, Changing and Building a Design in Vivado

Introduction

In this workshop you start with a small design that already works, and you take it through the same steps you would use on a real project: read it, change it, look at the hardware it describes, reorganize it, and run it all the way to a bitstream.

The design is two 8-bit counters. They are the same except for one thing: how they reset. One has a synchronous reset and one has an asynchronous reset. Seeing that difference in the schematic is the main idea of the day.

Nobody needs a board today. You will run Synthesis, Implementation and Generate Bitstream on your own computer. At the end we will generate one final bitstream together.

By the end you will be able to:

- Read a SystemVerilog module and say which parts remember something and which only calculate.
- Instantiate a module and connect its ports.
- Open the RTL schematic and match it to the code.
- Explain the difference between a synchronous and an asynchronous reset.
- Move logic into a lower-level module and keep the top level clean.
- Run the Vivado flow from source code to bitstream, and read the timing and utilization results.

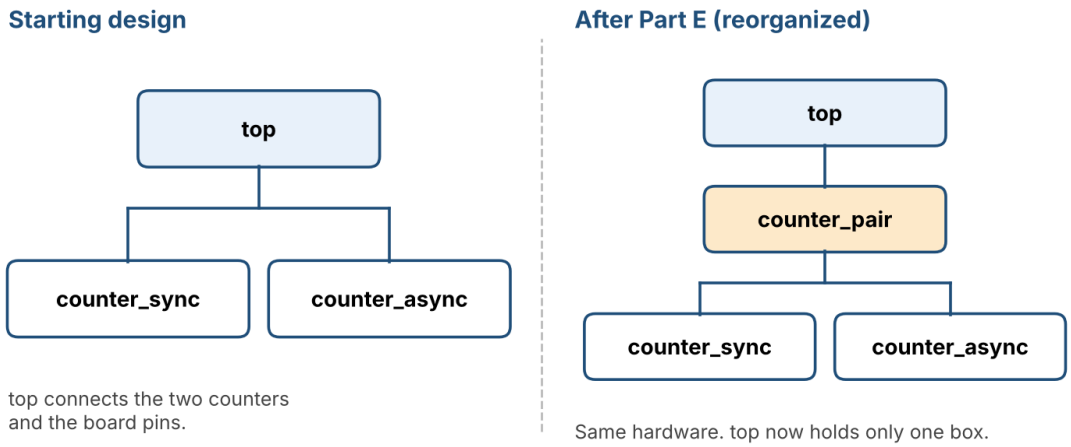


Figure 1. The hierarchy you start with (left) and the one you build in Part E (right).

Recall the rules we follow in this series:

- The top level module only instantiates and connects lower level modules. It has no logic of its own.

- Use one clock for the whole system. Every register uses `posedge clk`. Do not use another signal as a clock. Use an enable signal instead.
- The only other signal allowed next to `posedge clk` is an asynchronous reset.

Files you receive

File	What it is
`top.sv`	Top level. Has blanks for you to fill in (Part A).
`counter_sync.sv`	8-bit counter. Synchronous, active-high reset.
`counter_async.sv`	8-bit counter. Asynchronous, active-low reset. Has blanks for you to fill in (Part A).
`basys3.xdc`	Constraints file. Connects the ports to Basys 3 pins and sets the 100 MHz clock.

Procedure

Part A: Set up, read and fill in the blanks

1. Create a Vivado project named `Week1_Counters`. Add the three `.sv` files as design sources and `basys3.xdc` as a constraints file. When Vivado asks for a part, choose `xc7a35tcpg236-1` (the Basys 3). Click-by-click steps are in Appendix A.
2. Read `counter_sync.sv` first, from top to bottom, before you change anything. Then read `top.sv`. Write short answers to these questions in your notes:
   - Which signals are inputs and which are outputs of `counter_sync`?
   - Which line is the only place that remembers a value?
   - How many flip-flops does one counter use? Why?
   - What does `#(.W(8))` do when `top.sv` instantiates a counter?
   - What happens to `count` when `rst` is 1? When `en` is 0?
3. Fill in the blanks in `top.sv` and `counter_async.sv`. The comments next to each blank tell you what is missing.

counter_async.sv (student version)

```
// 8-bit counter with an ASYNCHRONOUS, active-low reset.
// Fill in the three blanks (________).
module counter_async #(
    parameter int W = 8
) (
    input  logic         clk,
    input  logic         rst_n,   // active-low, asynchronous
    input  logic         en,      // count only when en = 1
    output logic [W-1:0] count
);
    // Blank 1: an async reset must react when rst_n FALLS, not only on the clock.
    always_ff @(posedge clk or ________ rst_n) begin

        // Blank 2: rst_n is active-low, so we reset when it is 0.
        if (________)  count <= '0;

        // Blank 3: count only when enabled.
        else if (________) count <= count + 1'b1;
    end
endmodule
```

top.sv (student version)

```
// Top level: only instantiates modules and connects them.
// Fill in the blanks (________).
module top (
    input  logic       clk,
    input  logic       rst,        // reset for counter_sync  (active-high)
    input  logic       rst_n,      // reset for counter_async (active-low)
    input  logic       en,
    output logic [7:0] led_sync,
    output logic [7:0] led_async
);
    // Pattern:  module_name #(.PARAM(value)) instance_name ( .port(signal), ... );
    ________ #(.W(8)) u_sync (
        .clk   (________),
        .rst   (________),
        .en    (________),
        .count (________)
    );

    ________ #(.W(8)) u_async (
        .clk   (________),
        .rst_n (________),
        .en    (________),
        .count (________)
    );
endmodule
```

- **DO: Keep your finished `top.sv` and `counter_async.sv`. You will submit them.**

Part B: See the hardware

4. Open the RTL schematic: Flow Navigator, then RTL Analysis, then Open Elaborated Design, then Schematic. Click the plus sign on each counter to look inside. Find the flip-flops and the adder.
   - **DO: Copy and paste a snip of your top level schematic into your notes. Label it "before".**
   - **DO: Compare the flip-flop in `counter_sync` with the one in `counter_async`. What is different? Hint: look for an extra reset pin.**

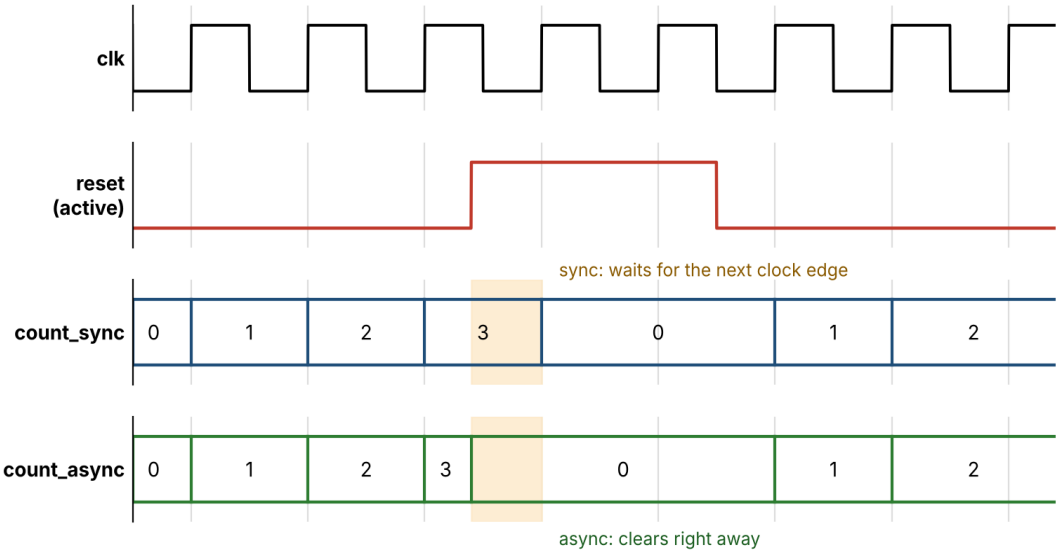


Figure 2. Reset goes active between two clock edges. The asynchronous counter clears immediately. The synchronous counter clears at the next edge.

   - **DO: Use Figure 2 to explain, in two sentences, why the two counters behave differently.**

Part C: Make one addition

5. Add a count direction input to `counter_sync`. Make three small edits:
   - In `counter_sync.sv`, add `input logic up,` to the port list. Change the counting line to `count <= up ? count + 1'b1 : count - 1'b1;`.
   - In `top.sv`, add `input logic up,` to the ports and connect it with `.up(up)` on `u_sync`.
   - In `basys3.xdc`, add a pin line for the new port. Use switch SW3. See Appendix B.
6. Run the elaborated schematic again.
   - **DO: Snip the new schematic. What new hardware appeared next to the adder? Why does a down counter need it?**

Part D: Two one-line changes

7. Reset type. In `counter_sync.sv`, change only the `always_ff` line to `always_ff @(posedge clk or posedge rst)`. Open the schematic again.
   - **DO: What changed on the flip-flop? Which counter does it now match?**

8. Enable polarity. In `counter_sync.sv`, change `else if (en)` to `else if (!en)`. Open the schematic again.
    - **DO: What new gate appeared on the enable path? Which switch position now makes the counter count?**
9. Undo the enable change from step 8 so the later parts match. Keep the reset change from step 7.

Part E: Reorganize the hierarchy

10. Create a new file `counter_pair.sv` with a module `counter_pair`. Move the two counter instances from `top` into it. Give it the ports `clk`, `rst`, `rst_n`, `en`, `up`, `count_sync` and `count_async`.
11. Change `top.sv` so it instantiates only `counter_pair`. Pass the ports straight through. Top level port names must stay the same, so the `.xdc` file does not change.
12. Look at the Hierarchy view in Sources. You should see `top`, then `counter_pair`, then the two counters. If not, right-click `top` and choose Set as Top.
    - **DO: Snip the Hierarchy view and the top level schematic. They should show the same hardware as before, with one extra level.**
    - **DO: Why is this a good idea when a design grows? Answer in one or two sentences.**

Part F: Implementation to bitstream

13. Click Run Synthesis. Wait for it to finish, then choose to run Implementation.
14. Click Generate Bitstream. If it fails with a DRC error about unconstrained or unplaced ports (NSTD-1 or UCIO-1), a top level port is missing a pin line in `basys3.xdc`. Fix it and run again.
15. Open the Project Summary. Find the Worst Negative Slack (WNS) and the Utilization chart. The clock period is 10 ns. The fastest clock your design could run at is

$$f_{max} \approx \frac{1}{T_{clk} - \text{WNS}}$$

For example, if WNS is 7.2 ns, then the smallest period is 10 − 7.2 = 2.8 ns, so the fastest clock is about 357 MHz.

- **DO: Snip the Project Summary. Show your calculation of the maximum clock frequency.**
- **DO: Which resource is used the most (LUT, FF or IO)? Why does this design use so many IO pins?**

Deliverables

By the end of the session, show an instructor:

- Your project reaching Generate Bitstream with no errors.
- Your notes with the answers and the snips from Parts B to F.
- A short explanation of Figure 2 in your own words.

Zip your `.sv` files and your `.xdc` file only. Do not include Vivado project folders.
Any team member may be asked to explain any part of the design.

Appendix A: Vivado steps

Create the project

- Open Vivado. Choose Create Project, then Next.
- Name it `Week1_Counters`. Pick a folder with no spaces in the path. Choose RTL Project.
- In Add Sources, click Add Files and select the three `.sv` files. Vivado reads `.sv` files as SystemVerilog.
- In Add Constraints, add `basys3.xdc`.
- In Default Part, choose the Parts tab and search for `xc7a35tcpg236-1`. Select it, then Finish.

Open the RTL schematic

- Flow Navigator: RTL Analysis, Open Elaborated Design, then Schematic. A new tab opens. Double-click a block (or click its plus sign) to see inside.

Set the top module

- Sources, Hierarchy tab. Right-click the module, then Set as Top. The top module is shown in bold.

Run the flow

- Flow Navigator: Run Synthesis. Wait for the progress bar in the top right to finish.
- Choose Run Implementation when asked, or click it in Flow Navigator.
- Click Generate Bitstream. When it finishes, you can skip Open Hardware Manager because there is no board.
- Open Project Summary (the first tab) to see WNS, utilization and power.

> **Tip**
>
> Synthesis and Implementation take longer on a bigger design. Use the RTL schematic for quick checks while you edit, and run the full flow only when the schematic looks right.

Appendix B: Pin constraints

Every top level port needs a line in the `.xdc` file that says which FPGA pin it uses. Without one, Generate Bitstream stops with a DRC error. The pins below are for the Basys 3.

Port	Pin	Basys 3 name
`clk`	W5	100 MHz clock
`rst`	V17	Switch SW0
`rst_n`	V16	Switch SW1
`en`	W16	Switch SW2
`up` (you add it)	W17	Switch SW3
`led_sync[7:0]`	U16, E19, U19, V19, W18, U15, U14, V14	LEDs 0 to 7
`led_async[7:0]`	V13, V3, W3, U3, P3, N3, P1, L1	LEDs 8 to 15

The line you add for `up` follows the same pattern as the others:

```
set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports up]
```